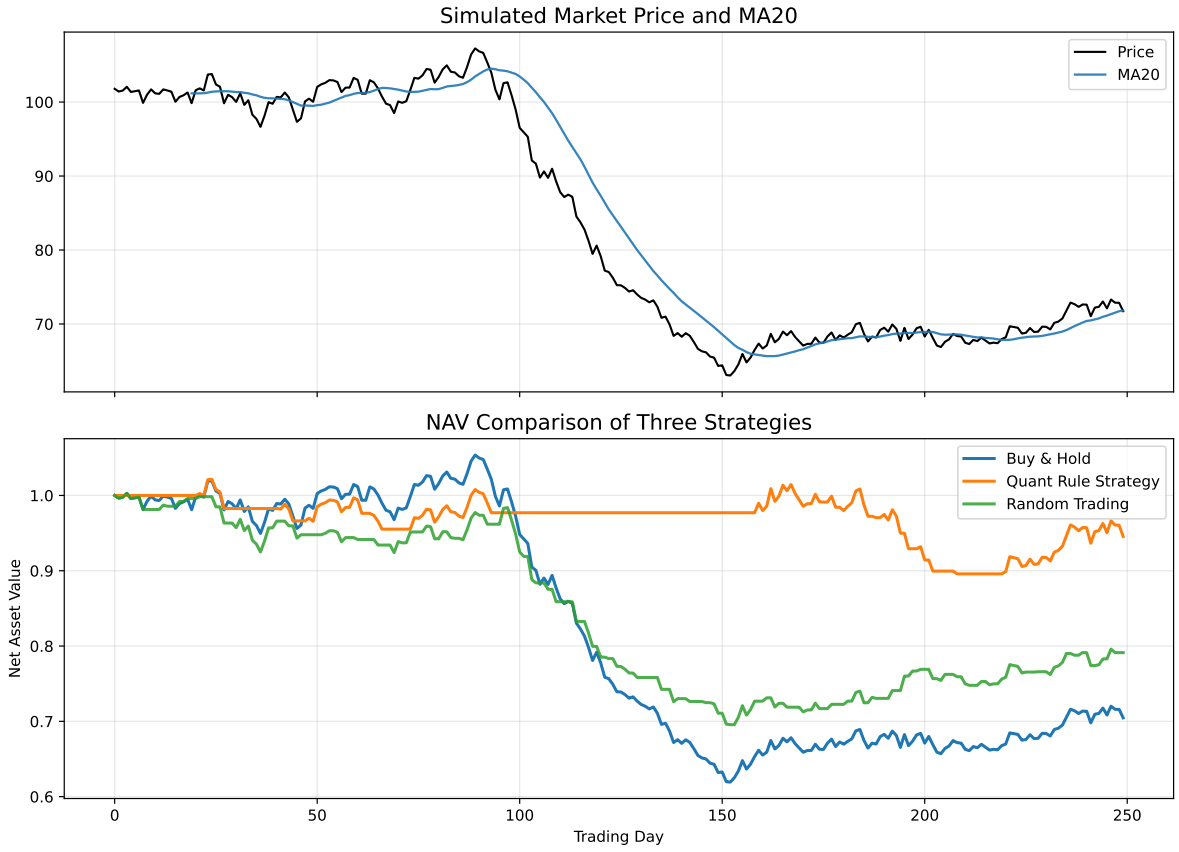
task1

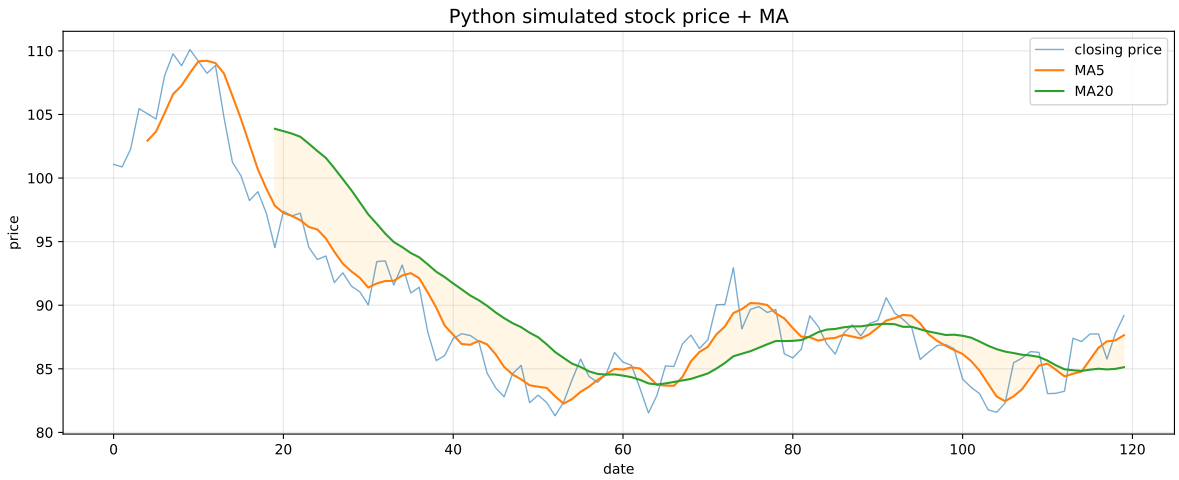
```
# ==========       ==========
import numpy as np  #
import pandas as pd  #      Excel
import matplotlib.pyplot as plt  #
import os
import warnings

warnings.filterwarnings("ignore", category=UserWarning, module="matplotlib")

plt.rcParams["axes.unicode_minus"] = False  #
plt.rcParams["font.sans-serif"] = ["DejaVu Sans", "Arial", "sans-serif"]  #
np.random.seed(7)  #

# ========== 1     →→ ==========
n1, n2, n3 = 90, 40, 120  #
ret = np.r_[  #
    np.random.normal(0.0010, 0.010, n1),  #  1
    np.random.normal(-0.012, 0.015, n2),  #  2
    np.random.normal(0.0012, 0.012, n3),  #  3
]  #
price = 100 * np.cumprod(1 + ret)  #  100    (1+  )

# ========== 2   DataFrame      ==========
df = pd.DataFrame({"close": price})  #        close
df["ret"] = df["close"].pct_change().fillna(0)  # pct_change =       0

# ========== 3   --  MA20    ==========
df["ma20"] = df["close"].rolling(20).mean()  # rolling(20).mean() = 20
```

```
df["signal_quant"] = (df["close"] > df["ma20"]).astype(int)  #    1  0

# ========== 4          ==========
rng = np.random.default_rng(7)  #      7
df["signal_random"] = rng.integers(0, 2, size=len(df))  #    0  1

# ========== 5              ==========
df["ret_quant"] = df["signal_quant"].shift(1).fillna(0) * df["ret"]  #
df["ret_random"] = df["signal_random"].shift(1).fillna(0) * df["ret"]  #
df["ret_buyhold"] = df["ret"]  #

# ========== 6           1  ==========
for col in ["ret_quant", "ret_random", "ret_buyhold"]:  #
    df[f"nav_{col}"] = (1 + df[col]).cumprod()  # (1+r)   =

# ========== 7  --  +      ==========
fig, axes = plt.subplots(2, 1, figsize=(11, 8), sharex=True)  # 2 1

axes[0].plot(df["close"], label="Price", color="black", linewidth=1.4)  #
axes[0].plot(df["ma20"], label="MA20", color="tab:blue", alpha=0.9)  # MA20
axes[0].set_title("Simulated Market Price and MA20", fontsize=14)
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(df["nav_ret_buyhold"], label="Buy & Hold", linewidth=2)  #
axes[1].plot(
    df["nav_ret_quant"], label="Quant Rule Strategy", linewidth=2
)  #
axes[1].plot(
    df["nav_ret_random"], label="Random Trading", linewidth=2, alpha=0.85
)  #
axes[1].set_title("NAV Comparison of Three Strategies", fontsize=14)
axes[1].legend()
axes[1].grid(True, alpha=0.3)
axes[1].set_xlabel("Trading Day")
axes[1].set_ylabel("Net Asset Value")

plt.tight_layout()  #
plt.show()  #  Notebook
```




```
# ==========    ==========
import numpy as np                          #
import pandas as pd                         #       Excel
import matplotlib.pyplot as plt             #

plt.rcParams["axes.unicode_minus"] = False  #

np.random.seed(42)           #
days = 120                   #    120
daily_returns = np.random.normal(loc=0.0008, scale=0.02, size=days)  #
price = 100 * np.cumprod(1 + daily_returns)     #  100

df = pd.DataFrame(
    {  #
        "closing price": price,                                  #
        "MA5": pd.Series(price).rolling(5).mean(),              #  5
        "MA20": pd.Series(price).rolling(20).mean(),            #  20
    }
```

```
)

plt.figure(figsize=(12, 5))                    #     12 5
plt.plot(df["closing price"], label="closing price", alpha=0.6, linewidth=1)
plt.plot(df["MA5"], label="MA5", linewidth=1.5)
plt.plot(df["MA20"], label="MA20", linewidth=1.5)
plt.fill_between(
    range(days), df["MA5"], df["MA20"], alpha=0.1, color="orange"
)  #
plt.title("Python simulated stock price + MA", fontsize=14)
plt.xlabel("date")
plt.ylabel("price")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Starting price: ¥{price[0]:.2f}")
print(f"Final price: ¥{price[-1]:.2f}")
print(f"Total return: {(price[-1] / price[0] - 1) * 100:.2f}%")
print(f"Minimum price: ¥{price.min():.2f}")
print(f"Maximum price: ¥{price.max():.2f}")
```



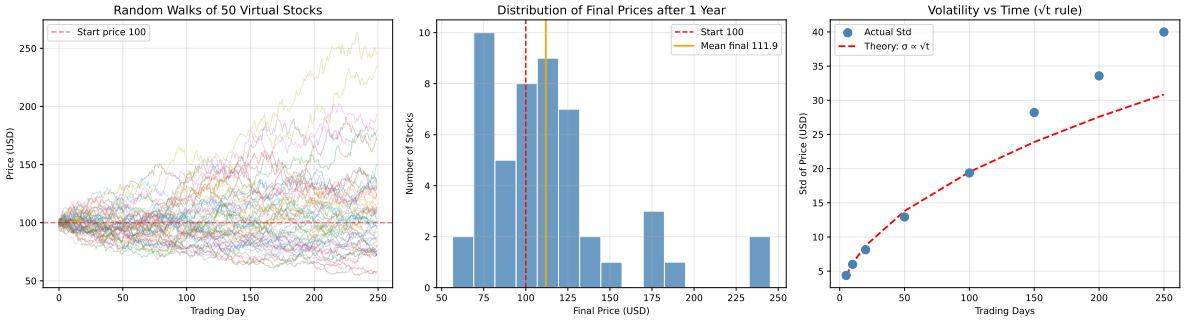

```
Starting price: ¥101.07
Final price: ¥89.18
Total return: -11.76%
Minimum price: ¥81.31
```

```
Maximum price: ¥110.10
```

Python

```python
# ==========      /    ==========
import numpy as np                        #
import matplotlib.pyplot as plt             #

plt.rcParams["axes.unicode_minus"] = False      #

np.random.seed(2026)        #

n_stocks = 50               #   50
n_days = 250                #    250
start_price = 100           #     100
daily_volatility = 0.02     #       2%

fig, axes = plt.subplots(1, 3, figsize=(18, 5))  # 1 3

# ---     50     ---
all_paths = []              #
for _ in range(n_stocks):   #   50
    daily_returns = np.random.normal(0, daily_volatility, n_days)  #
    price_path = start_price * np.cumprod(1 + daily_returns)       #
    all_paths.append(price_path)          #
    axes[0].plot(price_path, alpha=0.3, linewidth=0.8)  #

axes[0].axhline(y=start_price, color='red', linestyle='--', alpha=0.5, label='Start price 10
axes[0].set_title('Random Walks of 50 Virtual Stocks', fontsize=13)
axes[0].set_xlabel('Trading Day')
axes[0].set_ylabel('Price (USD)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# ---           ---
final_prices = [path[-1] for path in all_paths]   #
axes[1].hist(final_prices, bins=15, color='steelblue', edgecolor='white', alpha=0.8)  #
axes[1].axvline(x=start_price, color='red', linestyle='--', label=f'Start {start_price}')
```

```
axes[1].axvline(x=np.mean(final_prices), color='orange', linestyle='-', linewidth=2,
                label=f'Mean final {np.mean(final_prices):.1f}')
axes[1].set_title('Distribution of Final Prices after 1 Year', fontsize=13)
axes[1].set_xlabel('Final Price (USD)')
axes[1].set_ylabel('Number of Stocks')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# ---           ---
time_points = [5, 10, 20, 50, 100, 150, 200, 250]  #
std_at_time = []                                    #
for t in time_points:  #
    prices_at_t = [path[t-1] for path in all_paths]  #     t
    std_at_time.append(np.std(prices_at_t))         #

sqrt_time = np.sqrt(time_points)                    #
scale = std_at_time[0] / sqrt_time[0]               #

axes[2].scatter(time_points, std_at_time, s=80, color='steelblue', zorder=5, label='Actual St
axes[2].plot(time_points, scale * sqrt_time, 'r--', linewidth=2, label='Theory:    √t')
axes[2].set_title('Volatility vs Time (√t rule)', fontsize=13)
axes[2].set_xlabel('Trading Days')
axes[2].set_ylabel('Std of Price (USD)')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()                          #
plt.show()                                  #   Notebook

# ==========     ==========
print("=" * 60)
print("Experiment conclusions:")
print(f"  50 stocks all started at {start_price} USD")
print(f"  Highest final price: {max(final_prices):.2f} USD")
print(f"  Lowest final price:  {min(final_prices):.2f} USD")
print(f"  Average final price: {np.mean(final_prices):.2f} USD")
print(f"  Std of final prices: {np.std(final_prices):.2f} USD")
print("-" * 60)
print("  → Each individual path is unpredictable (left)")
print("  → But the ensemble of final prices forms a normal distribution (middle)")
print("  → Volatility grows with the square root of time (right) - Regnault's 1860s discovery
print("=" * 60)
```

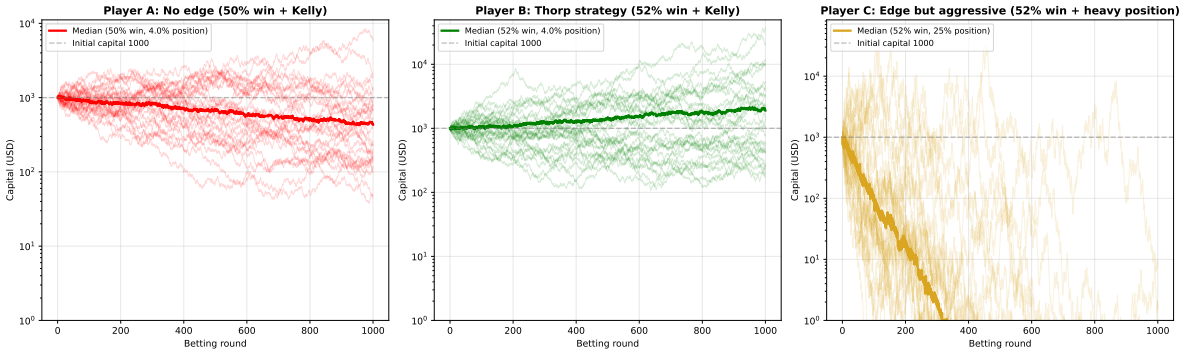



```
============================================================
Experiment conclusions:
  50 stocks all started at 100 USD
  Highest final price: 245.32 USD
  Lowest final price:  56.49 USD
  Average final price: 111.92 USD
  Std of final prices: 39.98 USD
------------------------------------------------------------
  → Each individual path is unpredictable (left)
  → But the ensemble of final prices forms a normal distribution (middle)
  → Volatility grows with the square root of time (right) - Regnault's 1860s discovery!
============================================================
```

```python
# ==========    --   +    ==========
import numpy as np                          #
import matplotlib.pyplot as plt               #

plt.rcParams["axes.unicode_minus"] = False      #

np.random.seed(2026)  #

n_rounds = 1000             #       1000
n_simulations = 200         #      200
initial_capital = 1000      #     1000

win_prob_no_edge = 0.50     #  A     50%
win_prob_edge = 0.52        #  B/C   2%
payout_ratio = 1.0          #    1:1
```

```
#   : f* = (bp - q) / b  →
kelly_fraction = (payout_ratio * win_prob_edge - (1 - win_prob_edge)) / payout_ratio  #
aggressive_fraction = 0.25  #  C      25%


def simulate_player(win_prob, bet_fraction, n_sims=n_simulations):  #
    """    n_rounds      """  #
    all_curves = []              #
    for _ in range(n_sims):              #
        capital = initial_capital        #
        curve = [capital]              #
        for _ in range(n_rounds):      #
            bet = capital * bet_fraction   #     =   ×
            if np.random.random() < win_prob:  #      →
                capital += bet * payout_ratio  #
            else:                              #   →
                capital -= bet                 #
            capital = max(capital, 0.01)       #
            curve.append(capital)  #
        all_curves.append(curve)  #
    return np.array(all_curves)        #     =  =


curves_A = simulate_player(win_prob_no_edge, kelly_fraction)       #    +
curves_B = simulate_player(win_prob_edge, kelly_fraction)          #    +
curves_C = simulate_player(win_prob_edge, aggressive_fraction)     #    +

fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))  #

configs = [  #
    (curves_A, 'Player A: No edge (50% win + Kelly)', 'red', f'50% win, {kelly_fraction*100:
    (curves_B, 'Player B: Thorp strategy (52% win + Kelly)', 'green', f'52% win, {kelly_frac
    (curves_C, 'Player C: Edge but aggressive (52% win + heavy position)', 'goldenrod', '52%
]

for ax, (curves, title, color, label) in zip(axes, configs):  #
    for i in range(min(30, n_simulations)):   #  30
        ax.plot(curves[i], alpha=0.15, linewidth=0.6, color=color)  #
    median_curve = np.median(curves, axis=0)  # 200
    ax.plot(median_curve, color=color, linewidth=2.5, label=f'Median ({label})')  #
    ax.axhline(y=initial_capital, color='gray', linestyle='--', alpha=0.5, label=f'Initial ca
    ax.set_title(title, fontsize=12, fontweight='bold')  #
```

```
    ax.set_xlabel('Betting round')  #
    ax.set_ylabel('Capital (USD)')  #
    ax.legend(fontsize=9, loc='upper left')  #
    ax.grid(True, alpha=0.3)  #
    ax.set_yscale('log')      #
    ax.set_ylim(1, None)  #

plt.tight_layout()                          #
plt.show()                                  #   Notebook

print("=" * 70)
print(f"Kelly formula: optimal f* = {kelly_fraction*100:.2f}% (win 52%, odds 1:1)")
print("=" * 70)

for name, curves, color in [("Player A (no edge)", curves_A, "red"),
                            ("Player B (Thorp)", curves_B, "green"),
                            ("Player C (aggressive)", curves_C, "goldenrod")]:
    finals = curves[:, -1]                      #
    win_rate = np.mean(finals > initial_capital) * 100  #
    median_final = np.median(finals)  #   median_final
    print(f"\n   {name} ")
    print(f"    Median final capital: {median_final:,.0f} USD")
    print(f"    Win probability: {win_rate:.1f}%")
    print(f"    Best case: {np.max(finals):,.0f} USD | Worst case: {np.min(finals):,.2f} USD

print("\n" + "=" * 70)
print("  → No edge, Kelly cannot save you (Player A)")
print("  → Just 2% win advantage + scientific position sizing = long-term compound growth (P
print("  → Edge with over-betting can lead to ruin (Player C)")
print("  → Thorp's core insight: probability edge × position sizing = quantitative profit fo
print("=" * 70)
```



```
======================================================================
Kelly formula: optimal f* = 4.00% (win 52%, odds 1:1)
======================================================================

  Player A (no edge)
    Median final capital: 432 USD
    Win probability: 23.5%
    Best case: 43,026 USD | Worst case: 13.27 USD

  Player B (Thorp)
    Median final capital: 1,897 USD
    Win probability: 70.5%
    Best case: 50,495 USD | Worst case: 90.58 USD

  Player C (aggressive)
    Median final capital: 0 USD
    Win probability: 0.0%
    Best case: 916 USD | Worst case: 0.01 USD

======================================================================
  → No edge, Kelly cannot save you (Player A)
  → Just 2% win advantage + scientific position sizing = long-term compound growth (Player B)
  → Edge with over-betting can lead to ruin (Player C)
  → Thorp's core insight: probability edge × position sizing = quantitative profit formula
======================================================================
```

× × =

```
 AAPL data loaded, total 124 trading days
   Date range: 2026-01-20  ~ 2026-07-17
   Latest close: $333.74
```

202.81

Figure 1: AAPL

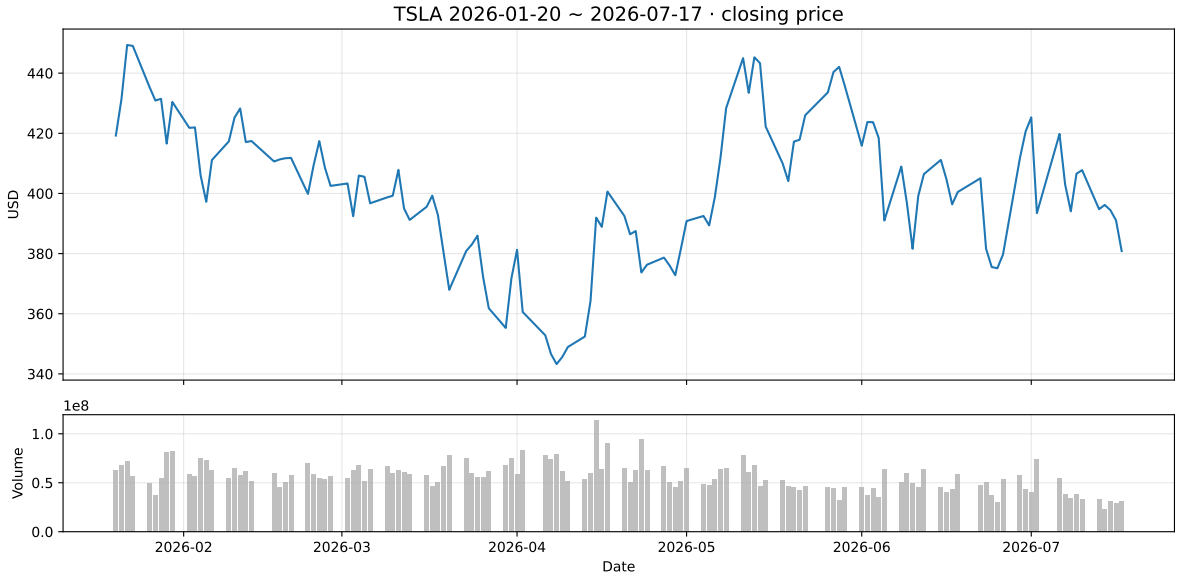
```
df_aapl = get_stock_data("NVDA")
plot_stock(df_aapl, symbol="NVDA")
```

```
NVDA data loaded, total 124 trading days
  Date range: 2026-01-20  ~ 2026-07-17
  Latest close: $202.81
```

380.84

```
df_aapl = get_stock_data("TSLA")
plot_stock(df_aapl, symbol="TSLA")
```

```
TSLA data loaded, total 124 trading days
  Date range: 2026-01-20  ~ 2026-07-17
  Latest close: $380.84
```


TSLA 2026-01-20 ~ 2026-07-17 · closing price
USD
440
420
400
380
360
340
1e8
Volume
1.0
0.5
0.0
2026-02
2026-03
2026-04
2026-05
2026-06
2026-07
Date